

Task 4: Trusted Profitability Model (SQL & Verification)

Objective:

To engineer a clean, aggregated data model at the 1:1 transaction grain. This model calculates the contribution economics (Gross Payment Volume, Revenue, and Costs) while actively neutralizing the one-to-many fan-out traps present in the operational tables (Events, Fraud, and Settlements) and avoiding temporal leakage.

Part 1: The SQL Data Model

*This query pre-aggregates the dimensional traps and joins them cleanly to the base transaction table, extracting the correct **SETTLEMENT** FX rate.*

```
WITH clean_transactions AS (  
  SELECT  
    transaction_id,  
    customer_id,  
    merchant_id,  
    route_id,  
    amount,  
    currency,  
    status,  
    DATE(created_at) AS txn_date,  
    STRFTIME('%Y-%m', created_at) AS txn_month  
  FROM "transaction"  
  WHERE amount > 0  
)
```

```
agg_fraud AS (  
  SELECT  
    transaction_id,  
    MAX(risk_score) AS max_risk_score  
  FROM fraud_decision  
  GROUP BY transaction_id  
)
```

```
agg_events AS (  
  SELECT  
    transaction_id,  
    SUM(processing_ms) AS total_processing_ms  
  FROM payment_event  
  GROUP BY transaction_id  
)
```

```
agg_settlement AS (  
  SELECT
```

```

SELECT
    transaction_id,
    SUM(settlement_amount) AS total_settlement_amount,
    SUM(fee_amount) AS total_settlement_fee
FROM settlement
GROUP BY transaction_id
),

agg_chargeback AS (
    SELECT
        transaction_id,
        SUM(amount) AS total_chargeback_amount
    FROM chargeback
    GROUP BY transaction_id
),

fx_settlement_rates AS (
    SELECT
        rate_date,
        currency,
        rate_to_usd
    FROM fx_rate
    WHERE rate_type = 'SETTLEMENT'
)

SELECT
    t.transaction_id,
    t.txn_date,
    t.txn_month,
    t.status,
    t.currency,
    t.amount AS amount_local,

    -- FX Conversion to USD
    (t.amount * fx.rate_to_usd) AS gpv_usd,

    -- Gross Revenue (Merchant Pricing Extraction)
    CASE
        WHEN m.pricing_plan = 'STANDARD_2.0%' THEN (t.amount * fx.rate_to_usd) * 0.020
        WHEN m.pricing_plan = 'ENTERPRISE_1.2%' THEN (t.amount * fx.rate_to_usd) *
0.012
        WHEN m.pricing_plan = 'CUSTOM_TIER' THEN (t.amount * fx.rate_to_usd) * 0.015
        ELSE 0
    END AS gross_revenue_usd,

    -- Processing Costs (Fixed + Variable)
    (r.fixed_fee + (t.amount * fx.rate_to_usd * r.variable_fee_pct)) AS
route_processing_cost_usd,

```

```

-- Attributed Operational Costs
COALESCE((s.total_settlement_fee * fx.rate_to_usd), 0) AS settlement_fee_usd,
COALESCE((cb.total_chargeback_amount * fx.rate_to_usd), 0) AS chargeback_loss_usd,

-- Final Contribution Profit (Only calculated for SUCCESSful and Settled txns)
CASE
  WHEN t.status = 'SUCCESS' AND s.total_settlement_amount IS NOT NULL THEN
    (
      (CASE
        WHEN m.pricing_plan = 'STANDARD_2.0%' THEN (t.amount * fx.rate_to_usd) *
0.020
        WHEN m.pricing_plan = 'ENTERPRISE_1.2%' THEN (t.amount *
fx.rate_to_usd) * 0.012
        WHEN m.pricing_plan = 'CUSTOM_TIER' THEN (t.amount * fx.rate_to_usd) *
0.015
        ELSE 0
      END)
      - (r.fixed_fee + (t.amount * fx.rate_to_usd * r.variable_fee_pct))
      - COALESCE((s.total_settlement_fee * fx.rate_to_usd), 0)
      - COALESCE((cb.total_chargeback_amount * fx.rate_to_usd), 0)
    )
  ELSE 0
END AS contribution_profit_usd,

-- Dimensional Attributes for Slicing
m.merchant_id,
m.pricing_plan,
m.category AS merchant_category,
m.country AS merchant_country,
c.segment AS customer_segment,
t.route_id,
r.provider AS route_provider,
f.max_risk_score,
e.total_processing_ms

FROM clean_transactions t
LEFT JOIN merchant m ON t.merchant_id = m.merchant_id
LEFT JOIN customer c ON t.customer_id = c.customer_id
LEFT JOIN route_cost r ON t.route_id = r.route_id
LEFT JOIN fx_settlement_rates fx ON t.currency = fx.currency AND t.txn_date = fx.rate_date
LEFT JOIN agg_fraud f ON t.transaction_id = f.transaction_id
LEFT JOIN agg_events e ON t.transaction_id = e.transaction_id
LEFT JOIN agg_settlement s ON t.transaction_id = s.transaction_id
LEFT JOIN agg_chargeback cb ON t.transaction_id = cb.transaction_id;

```

Part 2: Python Verification Script

This script was executed locally to mathematically verify that the SQL model did not suffer from data duplication (fan-out) and accurately applied the FX rate conversions.

```
import pandas as pd
import sqlite3

conn = sqlite3.connect('astrapay.db')

# Execute the final model to check row counts
df_model = pd.read_sql_query("SELECT * FROM trusted_profitability_model", conn)
df_txn_clean = pd.read_sql_query("SELECT * FROM 'transaction' WHERE amount > 0",
conn)

print("--- 1. Anti-Duplication Verification ---")
print(f"Original clean transactions: {len(df_txn_clean)}")
print(f"Transactions after joins: {len(df_model)}")
print(f"Are row counts identical?: {len(df_txn_clean) == len(df_model)}")

print("\n--- 2. Economics Math Verification ---")
sample_txn = df_model[df_model['transaction_id'] == 'TXN_10002'].iloc[0]
print(f"Transaction ID: {sample_txn['transaction_id']}")
print(f"Local Amount: {sample_txn['amount_local']} {sample_txn['currency']}")
print(f"GPV (USD) Applied: {sample_txn['gpv_usd']}")
```

Part 3: Verification Results

Output confirming structural integrity and mathematical accuracy:

```
--- 1. Anti-Duplication Verification ---
Original clean transactions: 197
Transactions after joins: 197
Are row counts identical?: True
```

```
--- 2. Economics Math Verification ---
Transaction ID: TXN_10002
Local Amount: 35.12 SGD
GPV (USD) Applied: 25.59152256
```